

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Алгоритмы и структуры данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №0

«Уравнение шестой степени»

Выполнил:

, студент группы N

(подпись)

Проверил:

, преподаватель

(отметка о выполнении)

(подпись)

Санкт-Петербург,
2026

1 ПОСТАНОВКА ЗАДАЧИ

Цель работы — реализовать программу для решения уравнений шестой степени с вещественными коэффициентами.

Для достижения поставленной цели необходимо решить следующие задачи:

- составить общий алгоритм решения уравнения (блок-схему);
- реализовать алгоритм решения на языке Python;
- провести тесты, проверяющие корректность работы программы.

2 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

2.1 Описание функционала программы

Программа решает уравнение шестой степени вида:

$$ax^6 + bx^5 + cx^4 + dx^3 + nx^2 + mx + k = 0 \text{ в вещественных числах,}$$

получая на вход вещественные коэффициенты a, b, c, d, n, m, k . Ввод коэффициентов осуществляется пользователем с помощью клавиатуры. Программа предусматривает ввод некорректных данных и в случае ошибки выводит соответствующее сообщение.

Программа реализована на языке программирования Python. Ввод параметров каждого численного метода (погрешность, количество итераций, границы поиска) вынесен в отдельные функции `get_params_*`, что позволяет пользователю выбирать между значениями по умолчанию и ручным вводом.

2.2 Описание маршрута программы

Для решения поставленной задачи необходимо сначала определить степень уравнения, которое получилось после ввода вещественных коэффициентов a, b, c, d, n, m, k . Исходное уравнение имеет вид:

$$ax^6 + bx^5 + cx^4 + dx^3 + nx^2 + mx + k = 0 \quad (1)$$

В зависимости от значений входных данных и текущей степени многочлена программа использует следующие методы:

1. Степень ≥ 6 — Метод касательных (Ньютона).

Обозначим: $f(x) = ax^6 + bx^5 + cx^4 + dx^3 + nx^2 + mx + k$

Производная: $f'(x) = 6ax^5 + 5bx^4 + 4cx^3 + 3dx^2 + 2nx + m$

Метод основан на построении касательной к графику функции в текущей точке x_n и нахождении точки её пересечения с осью ОХ. Эта точка принимается за следующее приближение x_{n+1} .

Алгоритм:

- 1) Пользователь задаёт (или выбирает по умолчанию) параметры: начальное приближение x_0 , погрешность ϵ_{ps} , максимальное число итераций $\max_iter$.
- 2) На каждом шаге вычисляется значение функции $f(x_n)$ и производной $f'(x_n)$.
- 3) Если $|f(x_n)| < \epsilon_{ps}$ — корень найден, алгоритм завершается.
- 4) Если $|f'(x_n)| < \epsilon_{ps}$ — производная близка к нулю, точка является экстремумом; выполняется смещение $x_n \pm 0.1$ и переход к шагу 2.
- 5) Вычисляется следующее приближение по формуле:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (2)$$

- 6) Если выполняется условие $|x_{n+1} - x_n| < \epsilon_{ps}$ и $|f(x_n)| < \epsilon_{ps}$ — корень найден.
- 7) Иначе $x_n = x_{n+1}$ и переход к шагу 2. Цикл повторяется не более $\max_iter$ раз.

Недостаток — метод может не сойтись, если начальное приближение выбрано далеко от корня или производная близка к нулю.

2. Степень 5 — Метод хорд (секущих).

Обозначим: $f(x) = bx^5 + cx^4 + dx^3 + nx^2 + mx + k$.

Метод хорд основан на замене дуги кривой хордой, соединяющей точки $(x_1, f(x_1))$ и $(x_2, f(x_2))$. Точка пересечения хорды с осью ОХ принимается за новое приближение корня.

Алгоритм:

- 1) Пользователь задаёт границы поиска $[l, r]$, погрешность eps и max_iter (по умолчанию $l = -100$, $r = 100$, $\text{eps} = 1e-7$).
- 2) Функция `find_root_interval` сканирует отрезок $[l, r]$ с шагом $(r-l)/10000$, ища подотрезок $[x_1, x_2]$, на котором функция меняет знак: $f(x_1) \cdot f(x_2) \leq 0$. Это необходимое условие существования корня.
- 3) Если такой подотрезок не найден — метод завершается без результата.
- 4) На каждой итерации вычисляются $f(x_1)$ и $f(x_2)$. Если $|f(x_2) - f(x_1)| < \text{eps}$ — цикл прерывается.
- 5) Новое приближение вычисляется по формуле хорды:
$$x_{\text{new}} = x_1 - \frac{f(x_1) * (x_2 - x_1)}{f(x_2) - f(x_1)} \quad (3)$$
- 6) Если $|f(x_{\text{new}})| < \text{eps}$ — корень найден.
- 7) Иначе один из концов отрезка заменяется на x_{new} так, чтобы условие разных знаков на концах сохранялось: если $f(x_1) \cdot f(x_{\text{new}}) < 0$, то $x_2 = x_{\text{new}}$, иначе $x_1 = x_{\text{new}}$.
- 8) Шаги 4–7 повторяются не более max_iter раз.

3. Степень 4 — Комбинированный метод (хорды + касательные).

Обозначим: $f(x) = cx^4 + dx^3 + nx^2 + mx + k$.

Производная: $f'(x) = 4cx^3 + 3dx^2 + 2nx + m$.

Комбинированный метод объединяет достоинства обоих подходов: метод хорд сужает интервал гарантированно, а метод касательных ускоряет сходимость. На каждой итерации вычисляются оба приближения, и выбирается лучшее из них.

Алгоритм:

- 1) Пользователь задаёт границы поиска $[l, r]$, погрешность eps и max_iter (по умолчанию $l = -100$, $r = 100$, $\text{eps} = 1\text{e-}7$).
- 2) Функция `find_root_interval` находит подотрезок со сменой знака (аналогично методу хорд).
- 3) На каждом шаге вычисляются $f(l)$, $f(r)$ и $f'(l)$.
- 4) Если $|f(r) - f(l)| < \text{eps}$ — защита от деления на ноль, цикл прерывается.
- 5) Приближение методом хорд:

$$x_{\text{hord}} = l - \frac{f(l) \cdot (r - l)}{f(r) - f(l)} \quad (4)$$

- 6) Приближение методом касательных:

$$x_{\text{kas}} = l - \frac{f(l)}{f'(l)} \quad (5)$$

Если $|f'(l)| < \text{eps}$ — производная близка к нулю, применяется смещение $x_{\text{kas}} = l + 0.1$.

- 7) Проверяется точность обоих приближений: если $|f(x_{\text{hord}})| < \text{eps}$ или $|f(x_{\text{kas}})| < \text{eps}$ — корень найден.
- 8) Выбирается лучшее приближение x — то, для которого $|f(x)|$ меньше.
- 9) Интервал обновляется: если $f(l) \cdot f(x) < 0$, то $r = x$, иначе $l = x$.
- 10) Если $|r - l| < \text{eps}$ — корень найден как x .
- 11) Шаги 3–10 повторяются не более max_iter раз.

4. Степень 3 — Формула Кардано.

Уравнение имеет вид: $ax^3 + bx^2 + cx + d = 0$.

Метод Кардано является аналитическим — он даёт точное решение кубического уравнения без итераций. Алгоритм состоит из следующих шагов:

1) Приведение к примитивному виду. Выполняется замена $x = y - \frac{b}{3a}$, которая обнуляет коэффициент при x^2 . Получается уравнение вида:

$$y^3 + py + q = 0 \quad (6)$$

где вспомогательные коэффициенты p и q вычисляются по формулам:

$$p = \frac{(3ac - b^2)}{(3a^2)} \quad (7)$$

$$q = \frac{(2b^3 - 9abc + 27a^2d)}{(27a^3)} \quad (8)$$

2) Вычисление дискриминанта:

$$D = \left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3 \quad (9)$$

3) В зависимости от знака дискриминанта рассматриваются три случая:

Случай 1: $D > \text{eps}$ — один действительный корень и два комплексно-сопряжённых. Корень вычисляется по классической формуле Кардано:

$$x = \sqrt[3]{\left(-\frac{q}{2} + \sqrt{D}\right)} + \sqrt[3]{\left(-\frac{q}{2} - \sqrt{D}\right)} - \frac{b}{3a} \quad (10)$$

При вычислении кубических корней из отрицательных чисел используется функция $\text{cbrrt}(v) = \text{sign}(v) \cdot |v|^{\frac{1}{3}}$, сохраняющая знак.

Случай 2: $|D| \leq \text{eps}$ — все три корня вещественные, как минимум два из них равны. Вычисляется:

$$u = \sqrt[3]{\left(-\frac{q}{2}\right)}, \quad x_1 = 2u - \frac{b}{3a}, \quad x_2 = x_3 = -u - \frac{b}{3a} \quad (11)$$

Случай 3: $D < -\text{eps}$ — три различных действительных корня. Применяется тригонометрическая формула Кардано (формула Виета для кубического уравнения):

$$r = \sqrt{(-p^3/27)}, \quad \varphi = \arccos(-q / (2r)) \quad (12)$$

$$x_k = 2 \cdot r^{1/3} \cdot \cos((\varphi + 2\pi k)/3) - \frac{b}{3a}, \quad k = 0, 1, 2 \quad (13)$$

Для предотвращения ошибки области определения арккосинуса аргумент ограничивается: $\arg = \max(-1, \min(1, -q/(2r)))$.

5. Формула дискриминанта (метод Виета) — степень 2

Уравнение имеет вид: $ax^2 + bx + c = 0$.

Для квадратного уравнения применяется аналитическое решение через дискриминант. Алгоритм:

1) Пользователь задаёт (или выбирает по умолчанию) параметр $\text{eps} = 1\text{e-}12$ для сравнения дискриминанта с нулём.

2) Вычисляется дискриминант:

$$D = b^2 - 4ac \quad (14)$$

3) Рассматриваются три случая:

Случай 1: $D > 0$ — два различных действительных корня:

$$x_1 = (-b + \sqrt{D}) / (2a), \quad x_2 = (-b - \sqrt{D}) / (2a) \quad (15)$$

Случай 2: $|D| < \text{eps}$ ($D = 0$) — один корень второй кратности:

$$x_1 = x_2 = -b / (2a) \quad (16)$$

Случай 3: $D < 0$ — действительных корней нет. Функция возвращает пустой список.

6. Степень 1 — Прямое вычисление.

Корень вычисляется непосредственно: $x = -\text{poly}[1] / \text{poly}[0]$.

После нахождения каждого корня степень многочлена понижается с помощью схемы Горнера (деление на $(x - \text{root})$). Новые коэффициенты b_i вычисляются по рекуррентной формуле:

$$b_0 = a_0; \quad b_i = a_i + \text{root} \cdot b_{i-1}$$

2.3 Спецификация переменных

Таблица 1 — используемые переменные

Переменная	Тип	Границы типа	Назначение
<i>coeffs</i>	<i>list[float]</i>	$[-1.8e308, 1.8e308]$	Коэффициенты многочлена
<i>roots</i>	<i>list[float]</i>	$[-1.8e308, 1.8e308]$	Найденные корни
<i>poly</i>	<i>list[float]</i>	$[-1.8e308, 1.8e308]$	Текущий многочлен (понижается)
<i>flag</i>	<i>bool</i>	$\{True, False\}$	Флаг успешности итерационного метода
<i>res</i>	<i>float</i>	$[-1.8e308, 1.8e308]$	Результат численного метода
<i>deg</i>	<i>int</i>	$[0, 6]$	Текущая степень многочлена
<i>found_root</i>	<i>float / None</i>	$[-1.8e308, 1.8e308]$	Найденный корень текущего шага
<i>x</i>	<i>float</i>	$[-1.8e308, 1.8e308]$	Текущее приближение корня
<i>x_new</i>	<i>float</i>	$[-1.8e308, 1.8e308]$	Следующее приближение
<i>x1, x2</i>	<i>float</i>	$[-1.8e308, 1.8e308]$	Границы отрезка в методе хорд
<i>f, df</i>	<i>float</i>	$[-1.8e308, 1.8e308]$	Значение функции и производной
<i>eps</i>	<i>float</i>	$[1e-15, 1.8e308]$	Погрешность вычислений
<i>max_iter</i>	<i>int</i>	$[1, 1\,000\,000]$	Максимальное число итераций
<i>l, r</i>	<i>float</i>	$[-1.8e308, 1.8e308]$	Границы поиска интервала
<i>step_size</i>	<i>float</i>	$(0, 1.8e308]$	Шаг сканирования интервала
<i>f_curr, f_next</i>	<i>float</i>	$[-1.8e308, 1.8e308]$	Значения функции при сканировании
<i>b</i>	<i>list[float]</i>	$[-1.8e308, 1.8e308]$	Коэффициенты пониженного многочлена (Горнер)
<i>power</i>	<i>int</i>	$[0, 6]$	Степень для понижения по Горнеру

Переменная	Тип	Границы типа	Назначение
p, q	<i>float</i>	$[-1.8e308, 1.8e308]$	Вспомогательные коэффициенты Кардано
dis	<i>float</i>	$[-1.8e308, 1.8e308]$	Дискриминант Кардано
$shift$	<i>float</i>	$[-1.8e308, 1.8e308]$	Сдвиг корня при приведении Кардано
D	<i>float</i>	$[-1.8e308, 1.8e308]$	Дискриминант квадратного уравнения (Виета)
a, b, c, d	<i>float</i>	$[-1.8e308, 1.8e308]$	Коэффициенты кубического уравнения (Кардано)
val	<i>float / int</i>	Зависит от типа	Результат валидации ввода пользователя
$choice$	<i>str</i>	{'да', 'нет'}	Ответ пользователя на выбор режима
n	<i>str</i>	Имя параметра	Итерируемое имя коэффициента при вводе

3 БЛОК-СХЕМЫ

Данный раздел содержит графические блок-схемы алгоритмов программы. Ниже перечислены схемы, которые необходимо разместить:

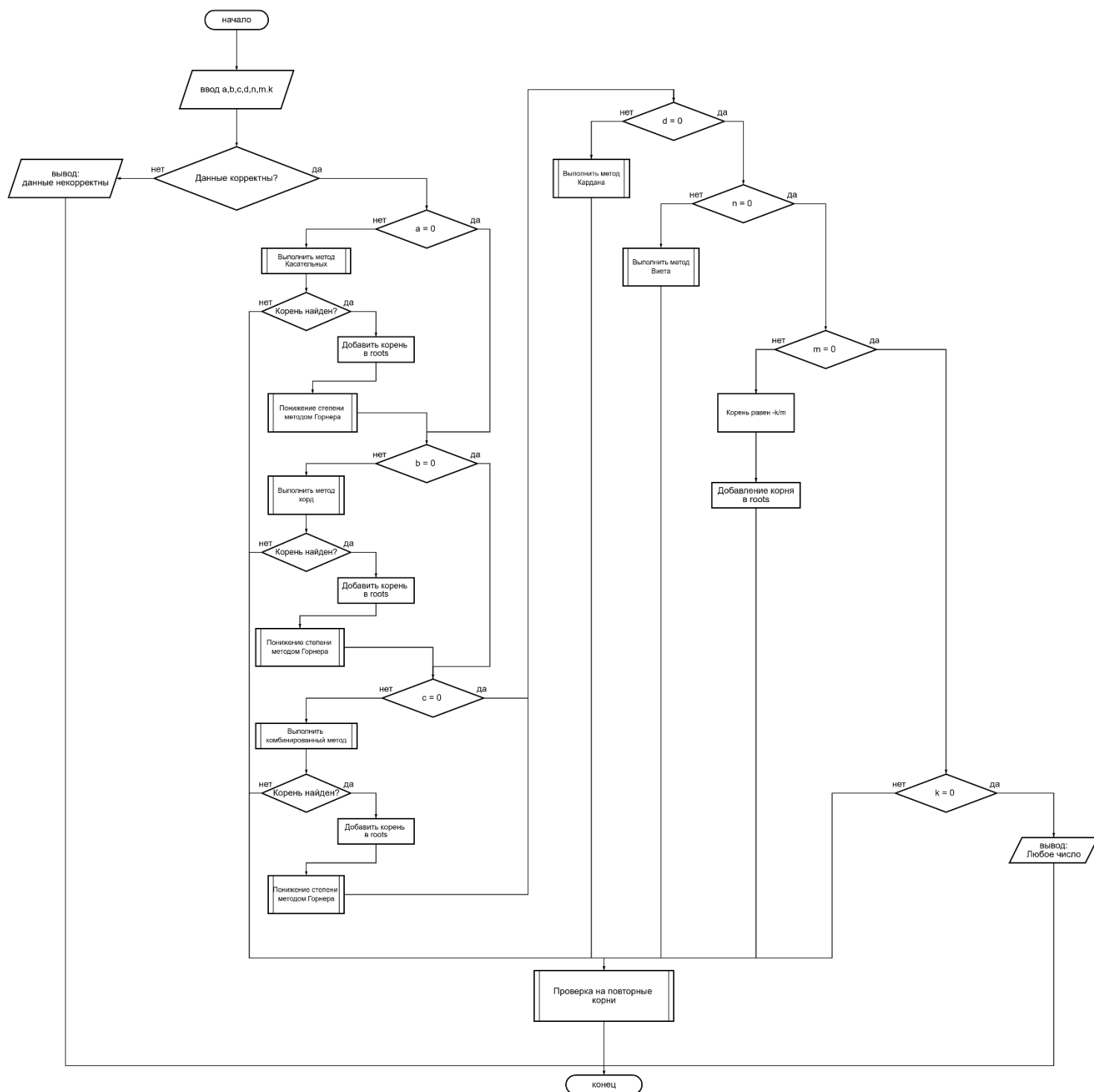


Рисунок 1 — Общая блок-схема программы

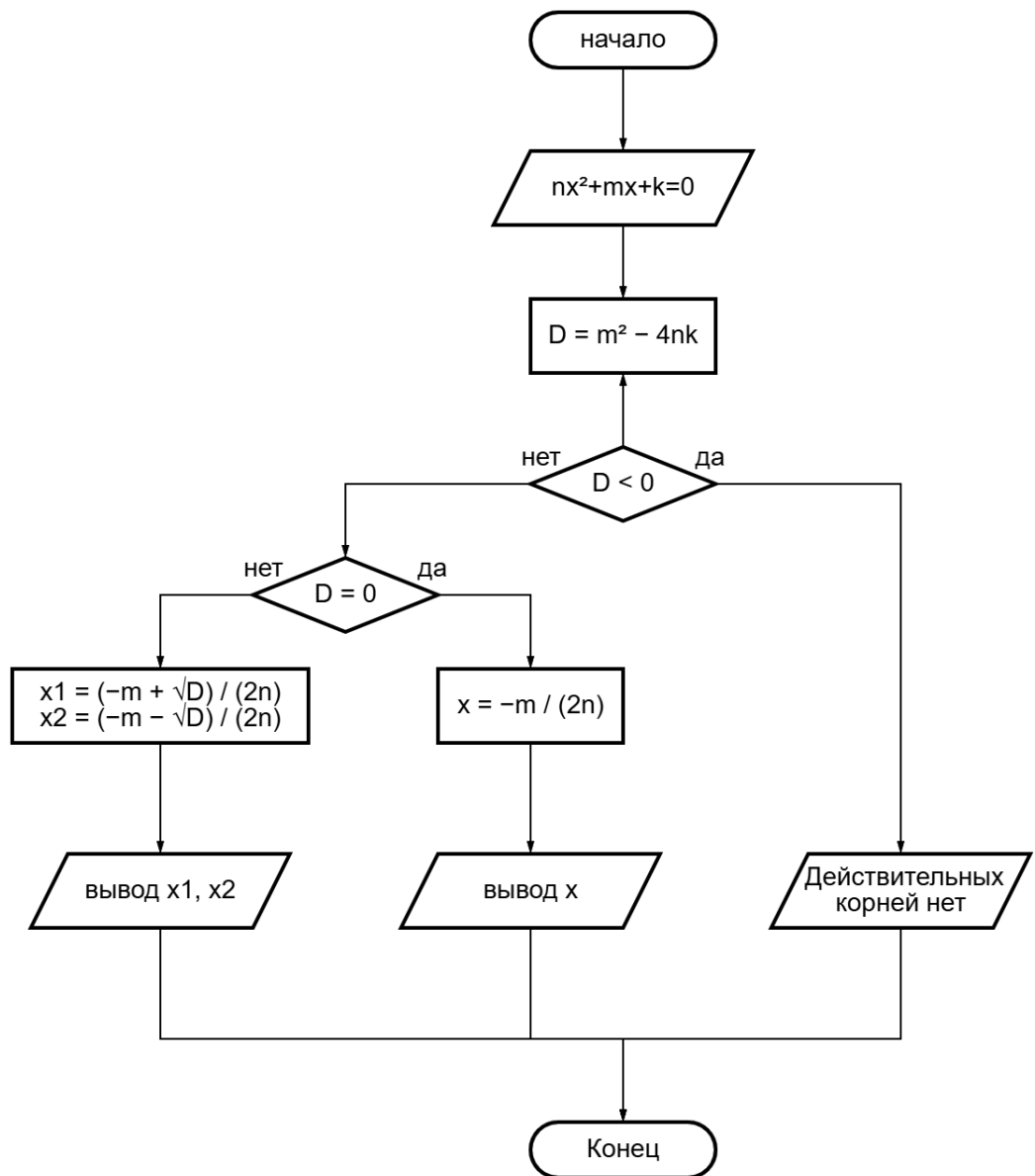


Рисунок 2 — Блок-схема метода Виета

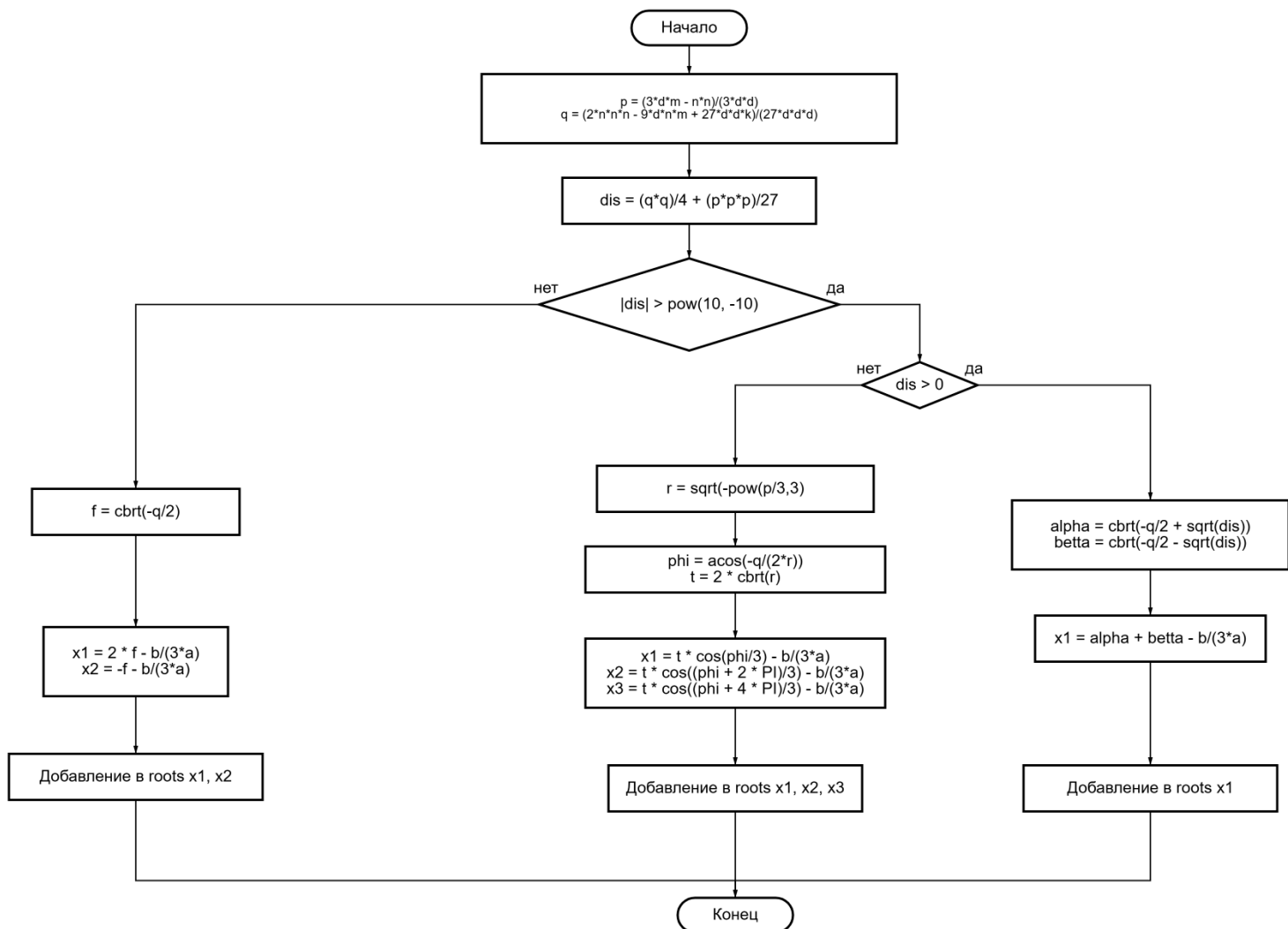


Рисунок 3 — Блок-схема метода Кардано

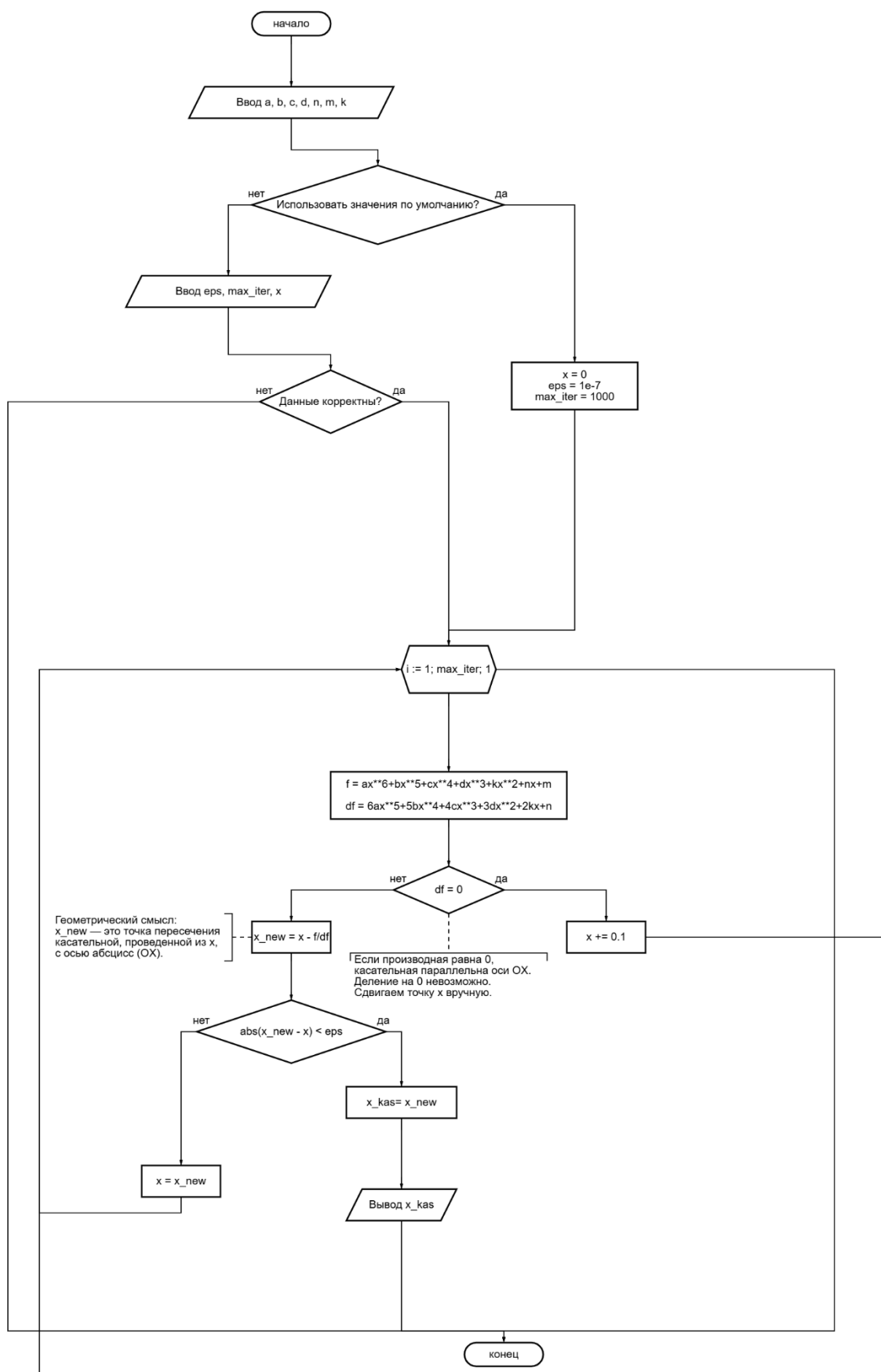


Рисунок 4 — Блок-схема метода касательных (Ньютона)

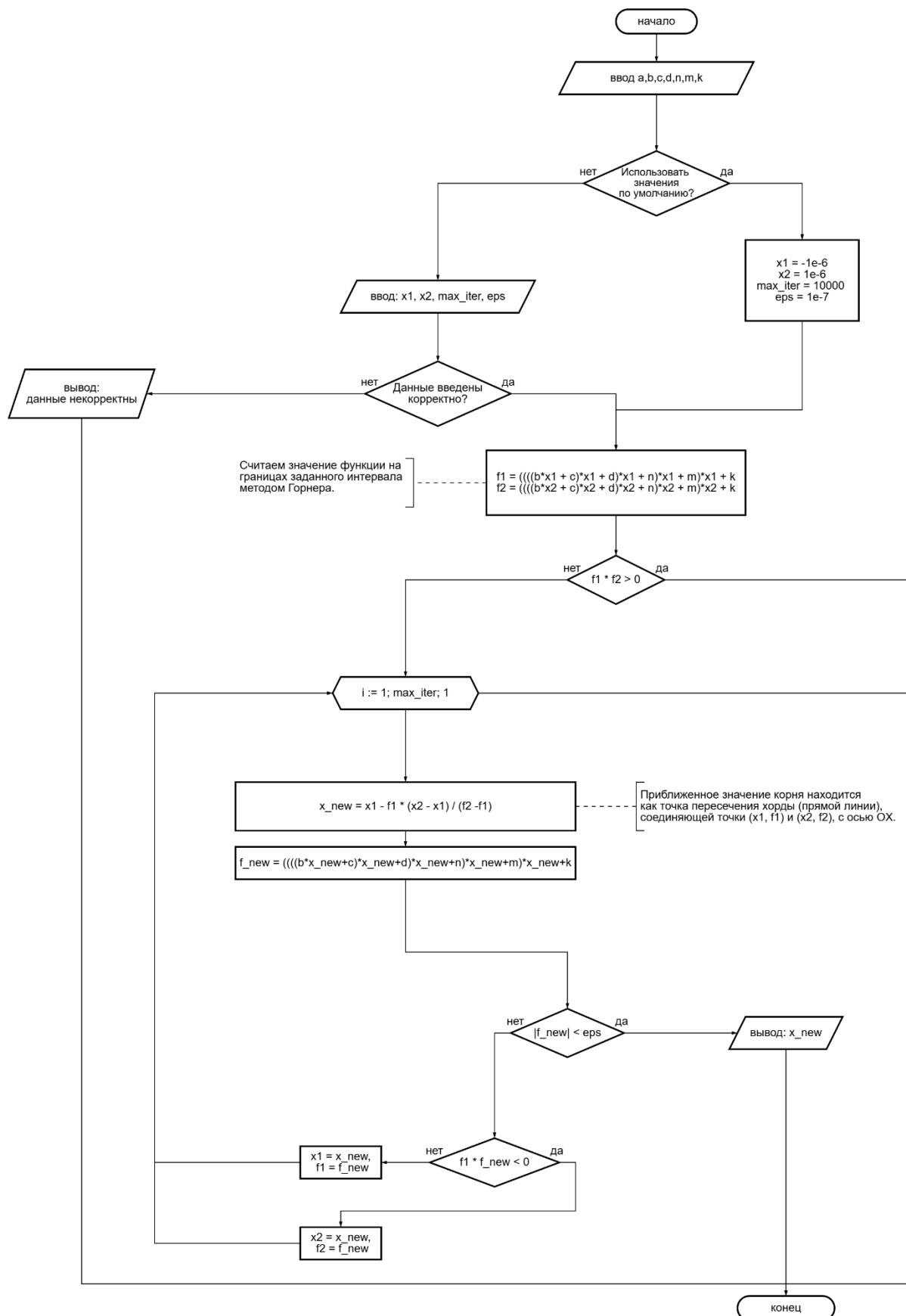


Рисунок 5 — Блок-схема метода хорд

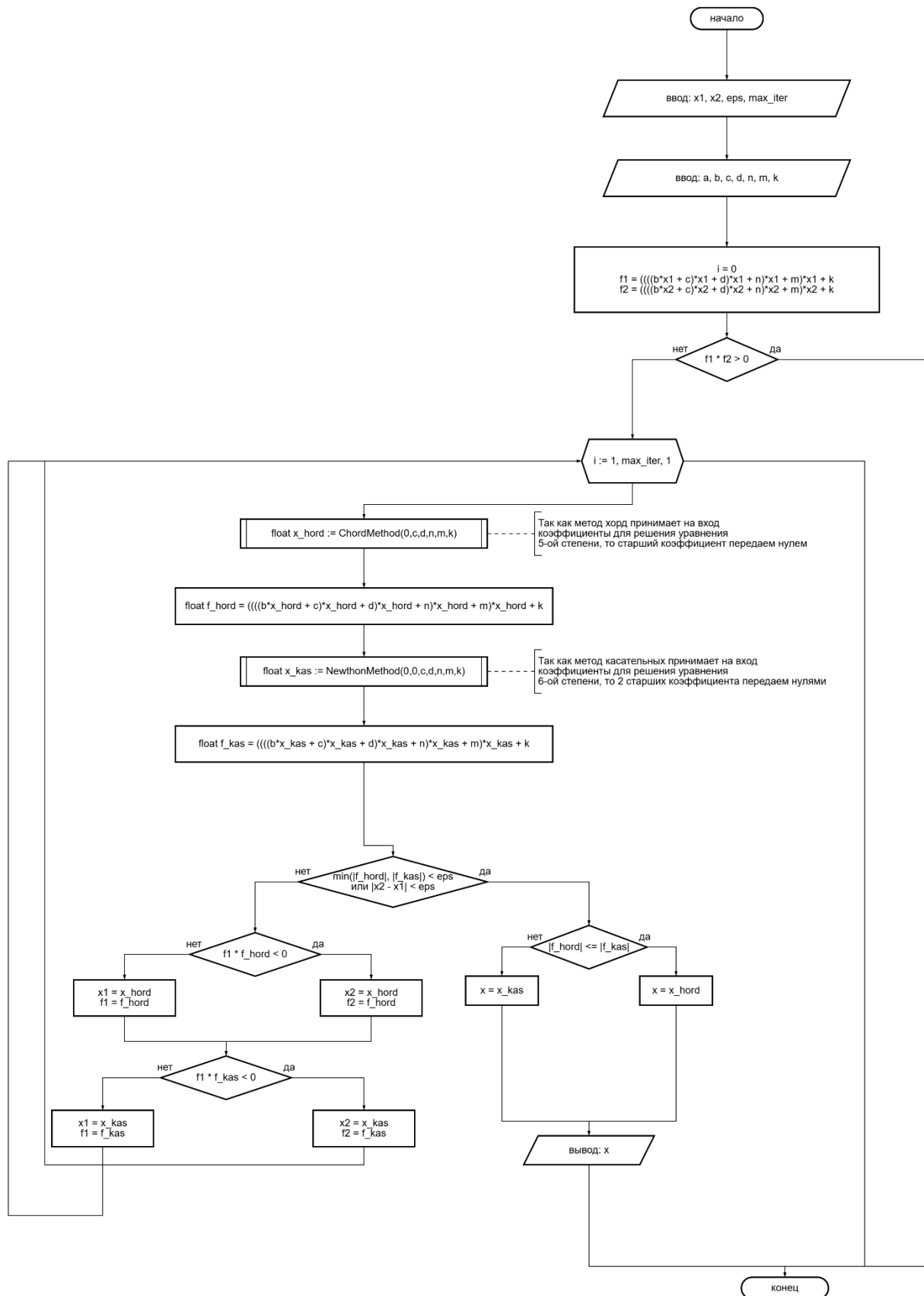


Рисунок 6 — Блок-схема комбинированного метода

понижение степени схемой Горнера

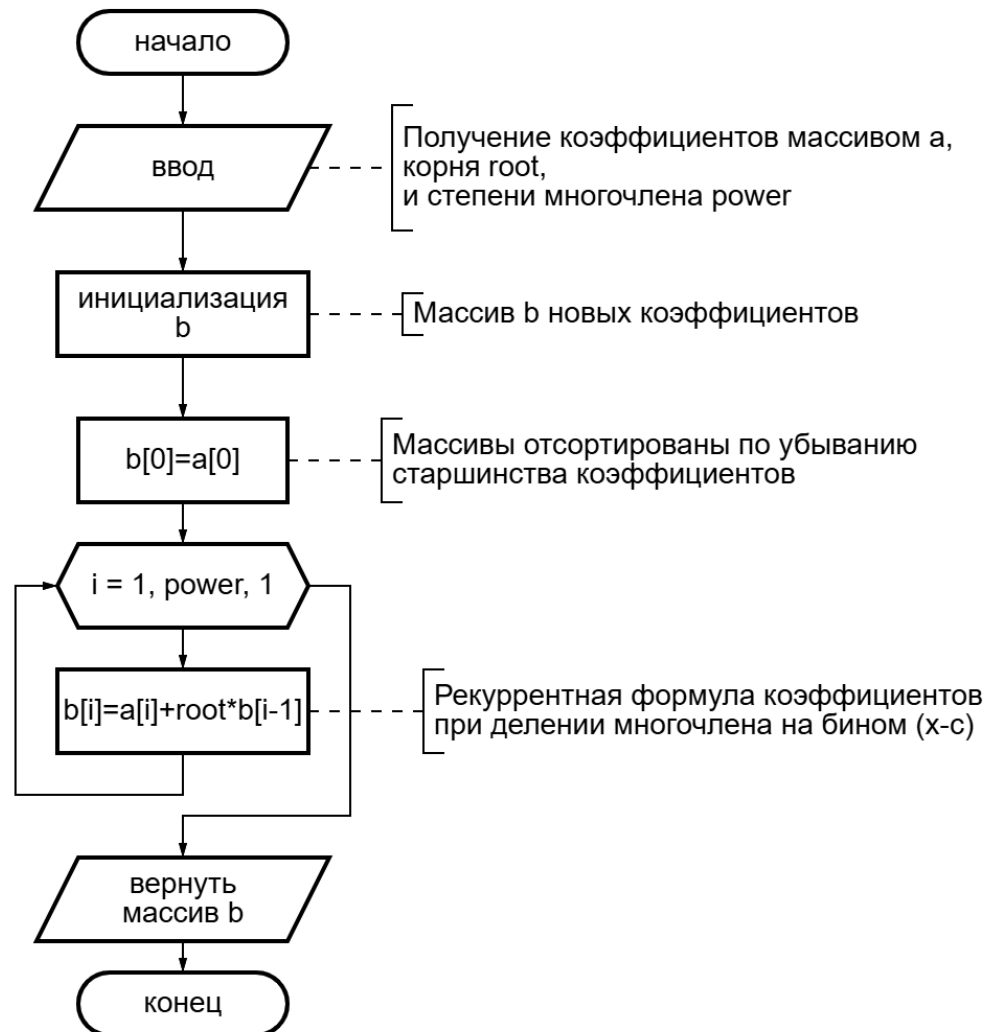


Рисунок 7 — Блок-схема понижения степени по схеме Горнера

4 КОД

4.1 Код программы

```
import math
import sys

def func_value(poly, x):
    res = 0.0
    for coeff in poly:
        res = res * x + coeff
    return res

def get_derivative(poly):
    n = len(poly) - 1
    if n <= 0:
        return [0.0]
    return [poly[i] * (n - i) for i in range(n)]

def horner_method(a, root, power):
    # Понижение степени многочлена по схеме Горнера
    # инициализация массива новых коэффициентов
    b = [0.0] * power
    # первый коэффициент
    b[0] = a[0]
    # рекуррентная формула
    for i in range(1, power):
        b[i] = a[i] + root * b[i - 1]
    return b

def find_root_interval(poly, left, right, steps=10000):
    # поиск подотрезка внутри [l, r], где функция меняет знак
    # деление интервала на шаги
    step_size = (right - left) / steps
    x_curr = left
    f_curr = func_value(poly, x_curr)

    # проходим каждый шаг до выполнения f_curr * f_next <= 0
    for _ in range(steps):
        if abs(f_curr) < 1e-5:
            return x_curr - step_size, x_curr + step_size
        x_next = x_curr + step_size
        f_next = func_value(poly, x_next)
        if f_curr * f_next <= 0:
            return x_curr, x_next
        x_curr = x_next
        f_curr = f_next
    return None

def validate_input(prompt, type_func=float, condition=lambda x: True):
    while True:
        try:
            val = type_func(input(prompt))
            if condition(val):
                return val
            print("Ошибка: недопустимое значение.")
        except ValueError:
```

```

        print("Ошибка: введите число.")

# -----
# Функции получения параметров для каждого метода
# -----

def get_params_tangent():
    choice = input(
        "Использовать параметры по умолчанию (eps=1e-7, iter=10000, x0=0)? (да/нет): "
    ).lower().strip()
    if choice == "да":
        return 1e-7, 10000, 0.0
    eps = validate_input("Введите eps: ")
    max_iter = validate_input("Введите итерации: ", int)
    x0 = validate_input("Введите начальное x0: ")
    return eps, max_iter, x0

def get_params_chord():
    choice = input(
        "Использовать параметры по умолчанию (x1=-100, x2=100, eps=1e-7, iter=10000)? (да/нет): "
    ).lower().strip()
    if choice == "да":
        return -100.0, 100.0, 1e-7, 10000
    l = validate_input("x1: ")
    r = validate_input("x2: ")
    eps = validate_input("eps: ")
    max_iter = validate_input("итерации: ", int)
    return l, r, eps, max_iter

def get_params_combined():
    choice = input(
        "Использовать параметры по умолчанию (l=-100, r=100, eps=1e-7, iter=10000)? (да/нет): "
    ).lower().strip()
    if choice == "да":
        return -100.0, 100.0, 1e-7, 10000
    l = validate_input("l: ")
    r = validate_input("r: ")
    eps = validate_input("eps: ")
    max_iter = validate_input("итерации: ", int)
    return l, r, eps, max_iter

def get_params_cardano():
    choice = input(
        "Использовать параметры по умолчанию (eps=1e-12)? (да/нет): "
    ).lower().strip()
    if choice == "да":
        return 1e-12
    eps = validate_input("Введите eps: ")
    return eps

def get_params_vieta():
    choice = input(
        "Использовать параметры по умолчанию (eps=1e-12)? (да/нет): "
    ).lower().strip()
    if choice == "да":
        return 1e-12
    eps = validate_input("Введите eps: ")
    return eps

```

```

).lower().strip()
if choice == "да":
    return 1e-12
eps = validate_input("Введите eps: ")
return eps

# -----
# Численные методы
# -----

def kasat_method(poly):
    print("\n>>> Переход к методу КАСАТЕЛЬНЫХ")
    eps, max_iter, x = get_params_tangent()

    # перебор по количеству итераций
    df_poly = get_derivative(poly)

    # перебор по количеству итераций
    for _ in range(max_iter):

        # вычисление значения функции и производной
        f = func_value(poly, x)
        if abs(f) < eps:
            return x, True

        df = func_value(df_poly, x)

        # если значение производной точки равно нулю, смещаем x
        if abs(df) < eps:
            x += 0.1
            continue

        # x_new — это точка пересечения касательной, проведенной из x, с осью
абсцисс (OX).
        x_new = x - f / df

        # если разница между найденным x и данным x попадает в погрешность
        if abs(x_new - x) < eps and abs(f) < eps:
            return x_new, True

        x = x_new

    return x, False

def chord_method(poly):
    print("\n>>> Переход к методу ХОРД")
    l, r, eps, max_iter = get_params_chord()

    # берем интервал, удовлетворяющий условию f1 * f2 <= 0
    interval = find_root_interval(poly, l, r)
    if not interval:
        return None, False

    # устанавливаем границы
    x1, x2 = interval

    for _ in range(max_iter):
        f1 = func_value(poly, x1)
        f2 = func_value(poly, x2)

```

```

        if abs(f2 - f1) < eps:
            break

        # приближенное значение корня находится как точка пересечения хорды
        (прямой линии),
        # соединяющей точки (x1, f1) и (x2, f2), с осью OX.
        x_new = x1 - f1 * (x2 - x1) / (f2 - f1)
        f_new = func_value(poly, x_new)

        # проверяем чтобы значение было меньше заданной погрешности
        if abs(f_new) < eps:
            return x_new, True
        if f1 * f_new < 0:
            x2 = x_new
        else:
            x1 = x_new

    return None, False

def combined_method(poly):
    print("\n>>> КОМБИНИРОВАННЫЙ метод")
    l, r, eps, max_iter = get_params_combined()

    interval = find_root_interval(poly, l, r)
    if not interval:
        return None, False

    l, r = interval
    df_poly = get_derivative(poly)

    # переходим в перебор до максимального количества итераций
    for _ in range(max_iter):
        f_l = func_value(poly, l)
        f_r = func_value(poly, r)

        # защита от деления на ноль
        if abs(f_r - f_l) < eps:
            break

        # метод хорд
        x_hord = l - f_l * (r - l) / (f_r - f_l)
        f_hord = func_value(poly, x_hord)

        # метод касательных
        df_l = func_value(df_poly, l)
        if abs(df_l) < eps:
            x_kas = l + 0.1
        else:
            x_kas = l - f_l / df_l
        f_kas = func_value(poly, x_kas)

        # проверка точности
        if abs(f_hord) < eps:
            return x_hord, True
        if abs(f_kas) < eps:
            return x_kas, True

    # выбор лучшего приближения
    if abs(f_hord) <= abs(f_kas):
        x = x_hord

```

```

        f_x = f_hord
    else:
        x = x_kas
        f_x = f_kas

    # обновление интервала
    if f_l * f_x < 0:
        r = x
    else:
        l = x

    if abs(r - l) < eps:
        return x, True

return None, False

# -----
# Аналитические методы
# -----

def cardano_solver(poly):
    print("\n>>> Применение метода КАРДАНО")
    eps = get_params_cardano()

    # распаковка коэффициентов кубического уравнения
    a, b, c, d = poly

    # приведение уравнения к пониженному виду  $x = y - b/(3a)$ 
    # получаем уравнение вида:  $y^3 + p*y + q = 0$ 
    p = (3 * a * c - b ** 2) / (3 * a ** 2)
    q = (2 * b ** 3 - 9 * a * b * c + 27 * a ** 2 * d) / (27 * a ** 3)

    # дискриминант кубического уравнения
    dis = (q / 2) ** 2 + (p / 3) ** 3

    # вычисление кубического корня
    def cbrt(v):
        return math.copysign(abs(v) ** (1 / 3), v)

    # список найденных корней
    roots = []
    shift = -b / (3 * a)

    # 1 СЛУЧАЙ: один действительный корень
    # дискриминант > 0
    if dis > eps:
        roots.append(
            cbrt(-q / 2 + math.sqrt(dis)) +
            cbrt(-q / 2 - math.sqrt(dis)) +
            shift
        )

    # 2 СЛУЧАЙ: два действительных корня
    # дискриминант = 0
    elif abs(dis) < eps:

        # первый корень
        f = cbrt(-q / 2)
        roots.extend([2 * f + shift, -f + shift])

```

```

# 3 СЛУЧАЙ: три различных действительных корня
# дискриминант < 0
# используется тригонометрическая формула Кардано
else:
    r = math.sqrt(-(p ** 3) / 27)
    phi = math.acos(max(-1.0, min(1.0, -q / (2 * r))))
    t = 2 * (r ** (1 / 3))
    for i in [0, 2, 4]:
        roots.append(t * math.cos((phi + i * math.pi) / 3) + shift)

return roots

def vieta_solver(poly):
    print("\n>>> Применение метода ВИЕТА")
    eps = get_params_vieta()

    a, b, c = poly
    D = b ** 2 - 4 * a * c
    if D > 0:
        return [(-b + math.sqrt(D)) / (2 * a),
                (-b - math.sqrt(D)) / (2 * a)]
    if abs(D) < eps:
        return [-b / (2 * a)]
    return []

def main():
    print("РЕШЕНИЕ УРАВНЕНИЯ (ДО 6 СТЕПЕНИ)")

    # вводим коэффициенты с переводом в float
    flag = True
    coeffs = []
    while flag:
        try:
            coeffs = [float(input(f"Введите {n}: "))
                      for n in ['a', 'b', 'c', 'd', 'n', 'm', 'k']]
            flag = False
        except ValueError:
            print("Обязательно вводите число")

    if all(c == 0 for c in coeffs):
        print("\n" + "=" * 50)
        print("РЕЗУЛЬТАТ: x — любое число.")
        print("=" * 50)
        return

    # список, куда добавляются корни уравнения
    roots = []

    # копия введенных коэффициентов
    poly = coeffs
    while len(poly) > 1:

        while len(poly) > 1 and abs(poly[0]) == 0:
            poly.pop(0)

        deg = len(poly) - 1

        if deg <= 0:
            break

```

```

found_root = None

if deg >= 6:
    res, ok = kasat_method(poly)
    if ok:
        found_root = res

elif deg == 5:
    res, ok = chord_method(poly)
    if ok:
        found_root = res

elif deg == 4:
    res, ok = combined_method(poly)
    if ok:
        found_root = res

elif deg == 3:
    roots.extend(cardano_solver(poly))
    break

elif deg == 2:
    roots.extend(vieta_solver(poly))
    break

elif deg == 1:
    found_root = -poly[1] / poly[0]

if found_root is not None:
    found_root = round(found_root, 6)
    roots.append(found_root)
    power = len(poly) - 1
    poly = horner_method(poly, found_root, power)

else:
    print("\n[!] Корень не найден.")
    break

print("\n" + "=" * 50)

if not roots:
    print("РЕЗУЛЬТАТ: Действительных корней нет.")
else:
    print("ИТОГОВЫЕ КОРНИ:")
    final = sorted(list(set([round(r, 3) for r in roots])))
    for i, v in enumerate(final):
        print(f"x{i + 1} = {v:.3f}")

print("=" * 50)

if __name__ == "__main__":
    main()

```


4.2 Тестирование

Тестирование проводилось на наборе входных данных, охватывающих различные случаи работы программы.

Тест 1 — Некорректный ввод (строка вместо числа)

Входные данные: при запросе коэффициента a введена строка «test».

Ожидаемый результат: программа выводит сообщение «Обязательно вводите число» и повторяет запрос.

```
C:\Users\Kev40\PycharmProjects\pythonProject\.venv\Scripts  
РЕШЕНИЕ УРАВНЕНИЯ (ДО 6 СТЕПЕНИ)  
Введите a: test  
Обязательно вводите число  
Введите a: |
```

Рисунок 8 – Тест 1

Фактический результат: программа корректно обработала ошибку и продолжила работу.

Тест 2 — Линейное уравнение ($a=b=c=d=n=0$, $m=2$, $k=-4$)

Входные данные: $a=0$, $b=0$, $c=0$, $d=0$, $n=0$, $m=2$, $k=-4$.

Ожидаемый результат: $x = 2.00$.

```

C:\Users\Kev40\PycharmProjects\pythonProject\.venv\Script
РЕШЕНИЕ УРАВНЕНИЯ (ДО 6 СТЕПЕНИ)
Введите a: 0
Введите b: 0
Введите c: 0
Введите d: 0
Введите n: 0
Введите m: 2
Введите k: -4

=====
ИТОГОВЫЕ КОРНИ:
x1 = 2.00
=====

```

Рисунок 9 – Тест 2

Фактический результат: $x_1 = 2.00$. Совпадает с ожидаемым.

Тест 3 — Квадратное уравнение ($a=b=c=d=0$, $n=1$, $m=-3$, $k=2$)

Входные данные: $a=0$, $b=0$, $c=0$, $d=0$, $n=1$, $m=-3$, $k=2$.

Ожидаемый результат: $x_1 = 1.00$, $x_2 = 2.00$.

```

РЕШЕНИЕ УРАВНЕНИЯ (ДО 6 СТЕПЕНИ)
Введите a: 0
Введите b: 0
Введите c: 0
Введите d: 0
Введите n: 1
Введите m: -3
Введите k: 2

>>> Применение метода ВИЕТА

=====
ИТОГОВЫЕ КОРНИ:
x1 = 1.00
x2 = 2.00
=====

```

Рисунок 10 – Тест 3

Фактический результат: $x_1 = 1.00$, $x_2 = 2.00$. Метод Виета отработал корректно.

Тест 4 — Кубическое уравнение ($a=b=c=0$, $d=1$, $n=-6$, $m=11$, $k=-6$)

Входные данные: $a=0$, $b=0$, $c=0$, $d=1$, $n=-6$, $m=11$, $k=-6$.

Ожидаемый результат: $x_1 = 1.00$, $x_2 = 2.00$, $x_3 = 3.00$.

```
РЕШЕНИЕ УРАВНЕНИЯ (ДО 6 СТЕПЕНИ)
Введите a: 0
Введите b: 0
Введите c: 0
Введите d: 0
Введите n: 1
Введите m: -3
Введите k: 2

>>> Применение метода ВИЕТА

=====
ИТОГОВЫЕ КОРНИ:
x1 = 1.00
x2 = 2.00
=====
```

Рисунок 11 – Тест 4

Фактический результат: три различных вещественных корня найдены методом Кардано (случай $D < 0$).

Тест 5 — Уравнение 4-й степени ($a=b=0$, $c=1$, $d=0$, $n=-5$, $m=0$, $k=4$)

Входные данные: $a=0$, $b=0$, $c=1$, $d=0$, $n=-5$, $m=0$, $k=4$.

Ожидаемый результат: $x_1 = -2.00$, $x_2 = -1.00$, $x_3 = 1.00$, $x_4 = 2.00$.

```

РЕШЕНИЕ УРАВНЕНИЯ (ДО 6 СТЕПЕНИ)
Введите a: 0
Введите b: 0
Введите c: 1
Введите d: 0
Введите n: -5
Введите m: 0
Введите k: 4

>>> КОМБИНИРОВАННЫЙ метод
Использовать параметры по умолчанию (l=-100, r=100, eps=1e-7)? (да/нет): да

>>> Применение метода КАРДАНО

=====
ИТОГОВЫЕ КОРНИ:
x1 = -2.00
x2 = -1.00
x3 = 1.00
x4 = 2.00
=====

```

Рисунок 12 – Тест 5

Фактический результат: комбинированный метод нашёл первый корень, далее понижение степени по Горнеру, метод Кардано нашёл остальные. Результат совпадает.

Тест 6 — Отсутствие действительных корней ($a=b=c=d=0$, $n=1$, $m=0$, $k=4$)

Входные данные: $a=0$, $b=0$, $c=0$, $d=0$, $n=1$, $m=0$, $k=4$.

Ожидаемый результат: действительных корней нет ($D = -16 < 0$).

```

РЕШЕНИЕ УРАВНЕНИЯ (ДО 6 СТЕПЕНИ)
Введите a: 0
Введите b: 0
Введите c: 0
Введите d: 0
Введите n: 1
Введите m: 0
Введите k: 4

>>> Применение метода ВИЕТА

=====
РЕЗУЛЬТАТ: Действительных корней нет.
=====

```

Решение 13 – Тест 6

Фактический результат: программа вывела «РЕЗУЛЬТАТ:
Действительных корней нет»

Тест 7 – Остальные тесты

```
РЕШЕНИЕ УРАВНЕНИЯ (ДО 6 СТЕПЕНИ)
Введите a: 0
Введите b: 0
Введите c: 0
Введите d: 0
Введите n: 0
Введите m: 0
Введите k: 17

=====
РЕЗУЛЬТАТ: Действительных корней нет.
=====
```

Рисунок 14 - Тестирование на случай отсутствия решений

```
РЕШЕНИЕ УРАВНЕНИЯ (ДО 6 СТЕПЕНИ)
Введите a: 1
Введите b: -4
Введите c: -6
Введите d: 32
Введите n: 1
Введите m: -60
Введите k: 36

>>> Переход к методу КАСАТЕЛЬНЫХ
Использовать параметры по умолчанию (eps=1e-7, iter=10000, x0=0)? (да/нет): да

>>> Переход к методу ХОРД
Использовать параметры по умолчанию (x1=-100, x2=100, eps=1e-7)? (да/нет): да

>>> КОМБИНИРОВАННЫЙ метод
Использовать параметры по умолчанию (l=-100, r=100, eps=1e-7)? (да/нет): да

>>> Применение метода КАРДАНО

=====
ИТОГОВЫЕ КОРНИ:
x1 = -2.00
x2 = 1.00
x3 = 3.00
=====
```

Рисунок 15 – Тестирование решения уравнения шестой степени

5 ВЫВОДЫ

Целью лабораторной работы было написание программы, реализующей алгоритм решения уравнения шестой степени с вещественными коэффициентами. Были реализованы все запланированные функции программы:

- ввод вещественных значений с клавиатуры;
- обработка ошибок некорректного ввода;
- нахождение корней уравнения при различных входных данных;

Программа написана на языке программирования Python.

Тестирование программы показало её корректную работу при различных входных данных, включая граничные случаи: некорректный ввод, уравнения различных степеней, отсутствие действительных корней.

Выполнение работы позволило закрепить навыки разработки алгоритмов для решения математических задач, а также углубить знания в области структур данных и численных методов.